

Week 4: Quantum String Operations (Processing Stage)

Hybrid QML and Quantum String Algorithms for Document Extraction

Pushkar Kumar • Mentor: Potluri Krishna Priyatham

Deliverable as specified in the project brief: Functional Grover-based string search simulation; oracle for $O(\sqrt{N})$ search complexity.

1 Objective

Implement pattern matching using Grover-based logic: develop a quantum string matching algorithm to find structural identifiers such as document ID numbers, and build the oracle.

2 A Construction That Does Not Search

The starter implementation for this track, written in July, contained the following:

```
match_positions = [i for i in range(len(text) - len(pattern) + 1)
                   if text[i:i+len(pattern)] == pattern]
oracle = build_marked_state_oracle(n_qubits, match_positions)
```

The comparison executes in Python, on the first line, before a single qubit is allocated. The circuit is then constructed to flip the phase of positions it has already been handed.

Run it and it appears to work: the measured distribution peaks at the correct position and the amplification is textbook. It is a correct demonstration of Grover *amplification*. It is not string matching, and no complexity claim can rest on it, because the $O(N)$ classical scan has already happened.

This construction is common in Grover string-matching demonstrations, and it is easy to ship without noticing—the output is indistinguishable from a working search. Identifying it is the reason this week’s deliverable was rebuilt rather than extended.

3 An In-Circuit Comparator Oracle

The replacement receives only the text and the pattern. Two registers:

- **pos**: $\lceil \log_2(N - M + 1) \rceil$ qubits, holding candidate start positions in superposition.
- **win**: $M \times b$ qubits, where b is the alphabet width in bits, holding the window of text at the candidate position.

3.1 Four Stages

1. Window loading. For each concrete position i , multi-controlled writes load the bits of $\text{text}[i : i + M]$ into **win**, conditioned on **pos** = i :

$$|i\rangle|0\rangle \longrightarrow |i\rangle|\text{text}[i : i + M]\rangle \quad (1)$$

Only 1-bits require a gate, since the register starts at $|0\rangle$.

2. Pattern XOR. An X gate is applied to $\text{win}[j]$ wherever pattern bit j is 1. Because the pattern is classical and known at circuit-build time, this needs no controls. Afterwards the

register holds $\text{window} \oplus \text{pattern}$, which is all-zeros exactly when they match—converting “are these strings equal?” into “is this register all-zeros?”, a condition a single multi-controlled gate can test.

3. Phase marking. Multi-controlled gates trigger on all-ones, so the register is sandwiched between X layers: flip everything, test, flip back.

4. Uncomputation. Stages 2 and 1 are reversed. Both are built purely from X and multi-controlled X gates, each its own inverse, so the same routine serves both directions.

3.2 Why Uncomputation Is Mandatory

After stage 1, `win` is *entangled* with `pos`: state $|i\rangle$ is paired with the text at position i . The phase from stage 3 attaches to that joint state.

Stopping there and applying the diffusion operator to `pos` alone fails: the window acts as a which-path record, the amplitudes cannot interfere, and Grover amplifies nothing. Uncomputation returns `win` to $|0\rangle$ for every branch, disentangling it so the phase belongs to `pos` alone.

3.3 An Encoding Detail

Alphabet codes begin at 1, never 0. When $N - M + 1$ is not a power of two, the position register addresses more states than there are valid positions. Those unreachable states never trigger a controlled load, so their window remains all-zeros—which is exactly the pattern stage 3 marks as a match. Reserving code 0 eliminates this failure mode entirely.

4 Staged Verification

Correctness was established stage by stage rather than by observing a plausible final output, because a search returning the right answer for the wrong reason looks identical to one that works.

Table 1: Verification results.

Stage	Check	Result
1	<code>win</code> holds $\text{text}[i : i + M]$ for a probe position	pass
2	Exactly the true match positions have negative amplitude	pass
3	Uncomputation leakage (probability outside <code>win</code> = 0)	0.00
4	Full search returns a true match	pass, 95–96%

On a 16-character hexadecimal field with a 2-character pattern: 14 qubits, 3 Grover iterations, correct position recovered in 95.2% of shots.

The zero-leakage measurement is load-bearing beyond this week: it is what licenses the browser demonstration to simulate the position register alone and have that reduction be exact rather than approximate.

5 Over-Rotation

A common intuition is that more Grover iterations sharpen the result. They do the opposite: amplitude rotates *past* the target and back down. Success probability oscillates rather than converging, and the optimal count $\lfloor \frac{\pi}{4} \sqrt{N/M} \rfloor$ is a genuine optimum, not a minimum.

6 Resource Scaling

Grover provides $O(\sqrt{N})$ queries. But the window-loading stage must write the text into the circuit, and each candidate position requires its own controlled load.

Table 2: Measured oracle resources, hexadecimal alphabet, 2-character pattern.

N (characters)	Qubits	Depth	CX gates
8	13	1,857	1,082
16	14	6,383	3,428
32	15	16,159	8,570
64	16	39,558	20,904

Fitting a power law in log–log space gives a CX scaling exponent of **1.39**—*superlinear* in text length. The measured curve lies above a linear reference and nowhere near $\sqrt{N}$.

The $O(\sqrt{N})$ advantage does not survive. For searching stored, unstructured classical text that must be loaded into the circuit, the loading cost dominates the search saving. This is a known caveat in the literature; the contribution here is reproducing it by direct measurement rather than quoting $\sqrt{N}$ unqualified.

Grover’s advantage remains real where the text is already available as a quantum oracle or in QRAM; where the search space is *generated* by a function rather than stored, as in satisfiability or cryptographic preimage search; or where a loaded state is reused across many queries. None of these describes scanning a document just received.

7 A Second Practical Limit

A full 38-symbol alphabet requires 6 qubits per character, so even a short search window exceeds what is simulable on ordinary hardware. The pipeline therefore searches the hexadecimal-representable identifier field, which is the realistic target for structured-field lookup in any case.

Table 3: Alphabet width against window cost.

Alphabet	Bits per character	Qubits for an 8-character window
Binary (2 symbols)	1	8
Hexadecimal (16)	5	40
Full charset (38)	6	48

8 Deliverable Status

Complete. A functional Grover-based string search that genuinely searches, verified stage by stage with zero uncomputation leakage, with resource scaling measured rather than assumed.

Artifacts: `track_b_search/oracle/comparator_oracle.py`, `notebooks/03_grover_comparator.ipynb`

Self-test: `python track_b_search/oracle/comparator_oracle.py`